

A6：构建“语言模型训练与对比分析平台”

1. prompt

- 我将进行《自然语言处理》这门课的 A6 作业：Week 7 随堂 Vibe 实验：构建“语言模型训练与对比分析平台”。本次实验的目的是：请利用 AI 辅助编程工具（Vibe Coding 模式），开发一个集成传统统计模型、自定义 RNN 训练以及现代预训练模型的综合交互式 Web 系统。通过亲手训练模型与对比生成结果，直观理解不同架构的特点与评价指标。本次实验分为四个模块，我将分模块发送（请了解这个作业的总体要求，不需要回复我任何信息）。总体要求为：使用 Python 的 Streamlit 框架，结合 nltk（用于统计模型）、torch（用于 RNN 训练）和 transformers（用于预训练模型）库，构建一个包含四个功能模块（多标签页结构）的 Web 应用。

GPT-5.3-Codex-Medium：

已了解。

- 模块 1：n 元语言模型与数据平滑 (n-gram & Smoothing)
 1. 对应内容：n 元语法与数据平滑/加法平滑
 2. 具体要求：在 Streamlit 应用的第一个标签页中，实现基于统计的 n 元语言模型。
 - 提供一个文本域加载一段基础英文语料（如 NLTK 自带的 reuters 或手动输入的一段短文）。
 - 使用 nltk 库，基于该语料构建一个 Trigram（三元语法）（或 Bigram 等）模型。统计并记录词频。
 - 提供一个句子输入框。计算该输入句子在当前 Trigram 模型下的生成概率（联合概率）。
 - 关键功能：提供一个 Checkbox（复选框）以切换是否开启 加一平滑（Add-one / Laplace Smoothing）。
 - 当输入包含语料库中未出现的 Trigram（即零概率事件）时，展示平滑前后的概率计算结果对比。

GPT-5.3-Codex-Medium：

我会先快速检查当前工作区里是否已有 Streamlit 项目文件（如 app.py 和 requirements.txt），然后直接把你这次“模块 1：n-gram + 平滑”功能接入到第一个标签页，并做一次基础运行检查。

已完成模块 1 的实现，并已接入第一个标签页。

变更文件：

- app.py
- requirements.txt

你这次的 2.1-2.5 要求对应如下：

1. 2.1 训练语料输入

- 第一个标签页提供了语料来源切换（NLTK Reuters 示例语料 / 手动输入语料）与文本域。

2. 2.2 构建 n-gram 模型并统计词频

- 使用 nltk 加载 Reuters 语料（自动尝试下载）。
- 支持 Bigram 和 Trigram（默认 Trigram）。
- 后台统计 context 频次与 context->next word 频次，并在页面展示 n-gram 统计示例表格。

3. 2.3 句子联合概率计算

- 提供句子输入框。
- 对输入句子逐步计算条件概率并连乘，得到联合概率 $P(\text{sentence})$ 。

4. 2.4 Add-one 平滑开关

- 提供复选框切换是否开启加一平滑。
- 勾选后直接展示平滑版本的逐步概率与联合概率。

5. 2.5 零概率事件对比

- 自动检测未见 n-gram（零概率事件）。
- 页面展示未平滑与平滑后的概率对比，并列出发零概率的具体 context 和词。

补充说明：

- 已创建四标签页结构，模块 2/3/4 目前是占位，方便你继续分模块迭代。
- 静态检查通过，app.py 未发现语法错误。

你现在可直接运行：

1. 安装依赖：pip install -r requirements.txt
2. 启动应用：streamlit run app.py

• 模块 2：从零训练 RNN 语言模型 (Train your own RNN-LM)

1. 对应内容：循环神经网络语言模型与长距离依赖
2. 具体要求：在第二个标签页中，使用 PyTorch 实现一个简单的 RNN 语言模型（字符级或单词级均可，为了训练速度建议使用字符级 Character-level RNN）。
 - 提供一个文本框，让用户输入一段自定义的短语料（如一首英文小诗或名言）。
 - 提供几个滑动条（Slider）调整训练超参数：Hidden Size (隐藏层维度, 16-128), Epochs (训练轮数, 10-200), Learning Rate (学习率)。
 - 提供一个【开始训练】按钮。点击后，在后台动态构建数据集并使用 torch.nn.RNN 或 torch.nn.LSTM 进行自回归训练（即用前t-1个字符预测第t个字符）。在界面上实时更新 Loss 折线图（使用st.line_chart）。

- 训练完成后，提供一个输入框让用户输入“起始字符 (Seed)”，模型根据训练结果，自动生成一段定长（如 50 个字符）的连续文本。

GPT-5.3-Codex-Medium:

我会先读取当前的 app.py，在现有四标签页结构上把“模块 2：从零训练字符级 RNN-LM”完整接入第二个标签页，并保留你已完成的模块 1 不受影响。接着我会做一次错误检查，确认能运行。

模块 2 已完成并接入第二个标签页，且不影响你现有的模块 1。

已修改文件：

- app.py

与你要求的对应关系：

1. 2.1 自定义语料输入

- 第二标签页新增多行文本框，可输入英文短语料（默认内置一段英文诗句）。

2. 2.2 超参数滑动条

- 已提供 Hidden Size (16-128)、Epochs (10-200)、Learning Rate (0.001-0.1) 三个 Slider。
- 另外补充了模型类型切换 (RNN/LSTM)，便于课堂对比。

3. 2.3 开始训练 + 实时 Loss 折线

- 已提供“开始训练”按钮。
- 点击后动态构建字符级训练数据（前 t-1 预测第 t 个字符），并使用 PyTorch 训练。
- 训练过程中实时更新 st.line_chart 曲线、进度条和当前 epoch/loss。

4. 2.4 Seed 续写生成

- 训练完成后可输入 Seed，并按设定长度生成连续文本（默认 50 字符，可调）。
- 若未训练直接生成，会给出明确提示。

检查结果：

- app.py 静态检查通过，无语法错误。
- 模块 3：预训练架构对比 (Masked LM vs. Causal LM)
 1. 对应内容：GPT 单向自回归 与 BERT 掩码与双向注意力
 2. 具体要求：在第三个标签页中，对比 BERT 和 GPT-2 的不同生成机制。请使用 Hugging Face 的 transformers 库的 pipeline 功能。
 - BERT (Masked Language Modeling): 加载 bert-base-uncased。提供一个输入框，要求用户输入带有 [MASK] 标记的句子（例如：'The man went to the [MASK] to buy some milk.'）。输出 BERT 预测的 Top-5 候选词及其概率。

- GPT-2 (Causal Language Modeling): 加载 gpt2。提供一个前缀提示词 (Prompt) 输入框。模型根据该前缀，从左到右自回归生成后续的 20 个单词。
- 将两者的输入区域和输出结果在页面上并排（或上下）清晰展示。

GPT-5.3-Codex-Medium:

我将先读取当前 app.py 的最新内容，定位第三个标签页入口，然后一次性补上 BERT/GPT-2 的 pipeline 对比模块并接入页面布局。

模块 3 已完成，并接入第三个标签页。

已修改文件：

- app.py

按你的要求对应如下：

1. 2.1 BERT 掩码预测

- 使用 transformers 的 pipeline 加载 bert-base-uncased。
- 提供带 [MASK] 句子输入框。
- 点击按钮后输出 Top-5 候选词及概率（表格展示 rank、token、probability）。

2. 2.2 GPT-2 自回归生成

- 使用 pipeline 加载 gpt2。
- 提供 Prompt 输入框。
- 点击按钮后按从左到右续写，设置 max_new_tokens=20，生成后在文本框展示。

3. 2.3 并排展示

- 在第三标签页用 st.columns(2) 左右并排：左侧 BERT，右侧 GPT-2。
- 底部补充了机制差异说明，便于课堂观察对比。

额外处理：

- 增加了模型懒加载缓存 (st.cache_resource)，首次加载较慢，后续复用更快。
- 对输入合法性和推理异常做了提示，避免页面静默失败。
- 代码静态检查通过，无语法错误。
- 模块 4：语言模型评价 (Perplexity 困惑度计算)
 1. 对应内容：交叉熵与困惑度
 2. 具体要求：在第四个标签页中，实现基于 GPT-2 的文本困惑度 (Perplexity, PPL) 计算。
 - 提供一个多行文本输入框，允许用户输入多段不同的测试句子。
 - 编写代码，使用刚才加载的 gpt2 模型和其对应的 Tokenizer，计算每个输入句子的交叉熵损失 (Cross-Entropy Loss)。
 - 根据公式 $PPL = \exp(\text{Loss})$ 计算并输出每个句子的困惑度得分。
 - 使用表格展示输入的句子及其对应的 PPL 分数。

GPT-5.3-Codex-Medium:

我将把第四标签页从占位改为 GPT-2 困惑度计算模块，复用已加载的 GPT-2 管道中的 model/tokenizer 来计算每句交叉熵和 PPL，并输出表格。随后我会做一次静态检查。

模块 4 已完成并接入第四个标签页。

变更文件：

- app.py

对应需求落实情况：

1. 2.1 多行输入

- 在第四标签页新增多行文本框，支持每行一条测试句子。

2. 2.2 交叉熵损失计算

- 复用已加载的 GPT-2 管道，直接使用其 model 和 tokenizer。
- 对每个句子执行前向计算，使用 labels=input_ids 得到 Cross-Entropy Loss。

3. 2.3 困惑度计算

- 按公式 $PPL = \exp(\text{Loss})$ 逐句计算并输出。

4. 2.4 表格展示

- 结果以表格展示 sentence、cross_entropy_loss、ppl。
- 对过短句子增加提示，避免无效计算。

另外：

- 第四个标签页标题已改为 Perplexity 评估。
- 静态检查通过，当前无语法错误。

2. 实验总结

本次实验围绕“语言模型训练与对比分析平台”完成了从统计方法到神经网络再到预训练模型的完整实践。模块1通过 n-gram 与加一平滑直观呈现数据稀疏问题，帮助我理解概率分配机制。模块2在小语料上训练字符级 RNN/LSTM，结合 Loss 曲线与续写结果，验证了隐藏状态对序列历史信息的建模作用。模块3对比 BERT 的掩码预测与 GPT-2 的自回归生成，清晰体现双向建模与单向约束的差异。模块4通过交叉熵与困惑度评估句子可预测性，形成了“训练—生成—评价”闭环。整体上，我对不同架构的适用场景、优势与局限有了更具体的认识。

在困惑度实验中，语法完整、语义自然的句子通常得到更低的 PPL，而随机拼接、缺乏语法结构的句子 PPL 明显更高。原因是前者更符合训练语料中的统计规律，模型对下一词分布更确定；后者上下文约束弱，预测分布更分散，不确定性更大。结果说明 PPL 能有效反映语言模型对句子的建模质量。

2.1 模块 1 观察任务：输入一个在语料中未出现过的合理句子。观察未开启平滑时，句子联合概率是否直接归零（数据稀疏问题）。开启加一平滑后，观察课件 P22 公式中的 δ 是如何将概率余量分配给未见事件的。

- 输入未在语料中出现的合理句子后，未平滑的三元模型会因某个条件概率为 0，导致联合概率连乘后直接归零，这正是数据稀疏带来的典型现象。开启加一平滑后，公式给每个候选词都加上 δ （通常取 1），并相应扩大分母，把原先集中在已见事件上的部分概率质量均匀让渡给未见事件，因此零概率被抬升为极小非零值，句子可比较性明显增强。

2.2 模块 2 观察任务：使用一段具有明显规律的文本（如 "hello world hello world..."）进行训练。调整不同的 Hidden Size 和 Epochs，观察 Loss 曲线的下降速度，以及模型最终生成的文本是否成功学习到了输入序列的模式（体现课件 P43 提到的“隐藏层状态维护过去信息”的特性）。

- 以“hello world”循环文本训练时，Loss 在前几轮下降很快，随后趋于平稳。Hidden Size 较小（如 16）时收敛慢、生成结果易错字或断裂；增大到 64 后，曲线下降更顺、续写更稳定。Epochs 提高后，模型基本能连续复现“hello world”模式，说明隐藏层确实在累积并利用前文信息来预测后续字符。

2.3 模块 3 观察任务：观察 BERT 是如何利用 [MASK] 左右两侧的上下文进行填空的（体现课件 P69 的双向注意力机制）。对比 GPT-2 是如何严格按照从左到右的顺序进行文本续写的（体现课件 P58 的单向自回归约束）。

- 实验中，BERT 在预测 [MASK] 时会同时参考左右语境，因此给出的候选词通常与整句语义和语法都更贴合；当右侧线索改变时，预测结果也会明显变化。相比之下，GPT-2 续写只能依赖左侧已生成内容逐步展开，后文一旦偏离，误差会被连续放大。这一对比直观体现了双向建模的理解优势与单向自回归的生成约束。

2.4 模块 4 观察任务：分别输入一句语法正确、逻辑通顺的常规英文句子，以及一句单词随机拼凑的乱码句子。观察困惑度（PPL）数值的差异，验证课件 P107 中提到的“困惑度越小，语言模型对该句子的建模性能越好”。

- 实验中，语法完整、语义连贯的常规句子得到的 PPL 明显更低；而随机拼凑的乱码句子由于词序异常、搭配不合语法，PPL 显著升高。这个差异说明模型对前者的下一词分布更有把握，对后者则高度不确定。由此可以验证：困惑度越小，语言模型对句子的建模效果越好，结果与课件结论一致。